Cutis Glow: Design and Development of a Web- and Mobile-Based Beauty Clinic Management System with a Client-Server Architecture and Role-Based Access Control

Course: Client-Server Mobile Device Programming - Supervising Lecturer: Vitradisa Pratama, S.ST., M.T.

Dhiya'an Sani (230102036)
Department of Informatics Engineering
Universitas Muhammadiyah Bandung
Bandung, Indonesia
230102036@umbandung.ac.id

Dini Sri Ayu Priyono (230102040)
Department of Informatics Engineering
Universitas Muhammadiyah Bandung
Bandung, Indonesia
dinisriayu22@gmail.com

Hilmi Durroh Taqwiyah (230102060)
Department of Informatics Engineering
Universitas Muhammadiyah Bandung
Bandung, Indonesia
hilmidt29@gmail.com

Putri Leonita Cikal Buchori (230102108)
Department of Informatics Engineering
Universitas Muhammadiyah Bandung
Bandung, Indonesia
pputrileonita@gmail.com

***Abstract*—** Manual administration in beauty clinics often gives rise to various problems, such as unstructured queues, scheduling conflicts between doctors and patients, scattered treatment histories, and difficulty for management in monitoring service performance in real time [1][3]. This project develops Cutis Glow, a client-server-based beauty clinic management system consisting of a web dashboard built with the Laravel framework and a mobile application built with Flutter, connected through a REST API secured with Laravel Sanctum authentication. The system differentiates access for three roles—Admin, Doctor, and Patient—through Spatie Laravel Permission, so that each role only sees the modules relevant to it: the administrator manages master data and clinic operations, the doctor manages schedules and treatment/medical records, and the patient books consultations and views their own treatment history. With a three-tier architecture (presentation layer, application layer, and data layer) following the MVC pattern, this system is expected to improve the efficiency, transparency, and traceability of beauty clinic services compared to conventional manual handling, while also enabling administrators, doctors, and patients to access clinic information consistently across both web and mobile platforms.

***Keywords*—** *Beauty Clinic, Laravel, Flutter, REST API, Client-Server, Role-Based Access Control, MVC*

I. INTRODUCTION

A. Background

Beauty clinic administration is a crucial part of daily clinic operations because it directly affects patient satisfaction, doctor scheduling, and the accuracy of treatment records. Errors or delays in registration, queuing, or record-keeping can reduce patient trust, cause schedule conflicts between doctors and patients, and add to the administrative burden whenever data discrepancies must be tracked down and corrected manually.

In many beauty clinics, patient registration, treatment scheduling, and medical/treatment history are still processed manually—for example, through handwritten notebooks, phone bookings, or simple spreadsheets. This approach is prone to unstructured queues, double bookings, incomplete treatment histories, and difficulty accessing data quickly and in real time [1], [3]. As the number of patients and the variety of available treatments grow, the volume of data that must be recorded, validated, and archived also increases proportionally, making manual methods increasingly difficult to maintain and audit.

Several previous studies have developed web-based clinic and salon information systems using the Laravel framework, showing that such systems can improve the efficiency and accuracy of clinic administration and reduce manual record-keeping errors [1], [3]. In addition, Flutter-based mobile applications have been used to bring salon and clinic management directly to the devices of both administrators and customers [2], while a combined Laravel and mobile architecture has proven capable of supporting flexible cross-platform patient registration [4], [5]. These findings show that a well-designed digital clinic platform not only speeds up administrative work but also improves data traceability and reduces the risk of human error.

Based on this background, a beauty clinic management system is needed that can be accessed not only through a web dashboard by administrators and doctors, but also through a mobile application by patients, so that the management of treatment data, schedules, and medical history can be carried out more

efficiently, transparently, and flexibly, with access strictly separated according to each user's role. This need forms the basis for the development of Cutis Glow, which is designed with a client-server architecture and

B. Problem Statement

- How can a beauty clinic management system be designed to be accessible through both a web dashboard and a mobile application in an integrated manner?
- How can a client-server system architecture be designed that securely connects the Flutter mobile application and the Laravel backend through a REST API?
- How can role-based access control be provided for three different roles—Admin, Doctor, and Patient—so that each role can only access the modules and data relevant to it?

C. Objectives

- To build a web-based (Laravel) and mobile-based (Flutter) beauty clinic management system integrated through a REST API.
- To implement a client-server architecture with an MVC pattern on the backend to manage patient, doctor, service/treatment, schedule, booking, and medical record data.
- To implement role-based access control using Spatie Laravel Permission for the Admin, Doctor, and Patient roles.
- To evaluate whether the resulting system is able to reduce administrative burden and improve data consistency compared to paper- or spreadsheet-based manual clinic administration.

D. System Scope

The scope of this project covers the design and implementation of the Cutis Glow application, consisting of a Laravel-based web dashboard for the Admin and Doctor roles, as well as a Flutter-based mobile client intended primarily for the Patient role (with Admin/Doctor also able to access relevant mobile modules). Both clients communicate exclusively through a REST API protected by Laravel Sanctum. This scope covers the management of master data (services/treatments, doctors, schedules), patient registration and booking, recording of treatment/medical history, user administration and role-based access rights using Spatie Laravel Permission, as well as basic reporting for clinic administrators. Aspects outside this scope, such as integration with an external payment gateway or a pharmacy/inventory system, are directions for future development.

II. PROPOSED SOLUTION

role-based access control that keeps the web and mobile clients synchronized against a single, consistent data source.

To address the problems described in Section I, Cutis Glow is developed as a client-server-based beauty clinic management application consisting of two main parts: a Laravel-based web dashboard and a Flutter-based mobile application. These two parts are connected through a REST API so that data managed through either the web or the mobile application remains consistent and is stored in the same database, thereby eliminating the risk of data duplication or conflict that commonly occurs when separate, disconnected tools are used.

A. Web Dashboard (Laravel) — Admin & Doctor

- User authentication and role-based access management (Admin, Doctor, Patient) using Laravel Sanctum and Spatie Laravel Permission, so that each user only has access to the modules relevant to their role.
- A dashboard containing statistics on bookings, patients, and treatment performance, allowing administrators to monitor clinic activity at a glance.
- CRUD (Create, Read, Update, Delete) operations for Patient, Doctor, Service/Treatment, Schedule, and Booking data.
- A Doctor workspace for managing personal schedules, confirming bookings, and recording treatment/medical notes for each patient visit.
- User and Role Management for configuring system access rights at a granular, module-level scale.
- A REST API secured with Laravel Sanctum, provided for consumption by the mobile application.

B. Mobile Application (Flutter) — Patient-Oriented

- Login and splash screens as the application's entry point, including session handling so that authenticated users remain logged in across sessions.
- A dashboard summarizing available treatments, upcoming bookings, and clinic promotions.
- Browsing of Services/Treatments and Doctors, along with an online consultation booking/scheduling feature.
- Display of personal treatment history and medical records, connected to the backend through a REST API, allowing patients to review previous visits and recommendations.

C. Role-Based Access Control Strategy

Because Admin, Doctor, and Patient use the same backend and database, access control is centralized on the Laravel server using Spatie Laravel Permission, rather than being duplicated on each client. Each role is mapped to a specific set of access rights at the module level: the Admin has full access to master data, user/role management, and clinic-wide reporting; the Doctor is restricted to their own schedule, bookings assigned to them, and the medical/treatment records of patients they have handled; the Patient is restricted to their own profile, bookings, and treatment history. This design choice reduces the possibility of unauthorized data access between roles and simplifies future maintenance, since any change to access rules only needs to be applied once on the server side to be immediately reflected on both clients.

III. SYSTEM ARCHITECTURE

The Cutis Glow system architecture is designed using a three-tier client-server pattern, namely the presentation layer, the application layer, and the data layer, as illustrated in Fig. 1.

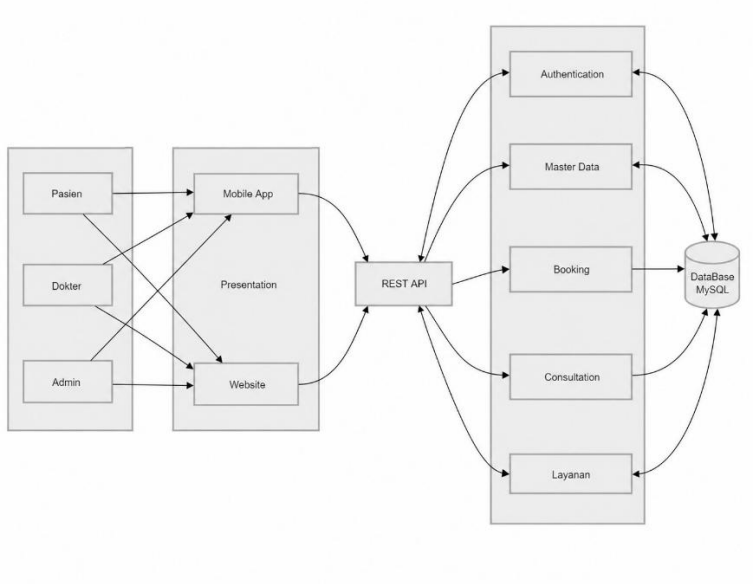

Image 1 Cutis Glow System Architecture (Client–Server, MVC, Role-Based Access Control).

Broadly speaking, the architectural flow shown in Fig. 1 works as follows. Admin, Doctor, or Patient users access the system through two types of clients: the Flutter mobile application (primarily for Patients, with Login/Splash, Dashboard, Service/Treatment browsing, Booking, and Treatment History modules) and the Laravel web application (for Admin and Doctor, with Dashboard, Patient Management, Schedule & Service Management, Medical/Treatment Records, and User & Role Management modules). Both clients communicate with the Application Layer (Laravel Backend) through the REST API and its associated API Routes.

At the application layer, the Model-View-Controller (MVC) pattern is applied, in which each incoming request is routed to the appropriate controller, namely the Auth, User & Role, Patient, Doctor/Schedule, Service/Treatment, Booking, and Medical Record controllers. Each controller interacts with the related Model to process business logic, which is then stored in and retrieved from the Database at the data layer.

This layered separation provides several benefits. First, the codebase becomes more structured and easier to maintain, since presentation logic, business logic, and data access are clearly separated. Second, it allows the mobile and web applications to consistently share a single data source, so that bookings made through the mobile application are immediately visible to doctors and administrators on the web dashboard. Third, it improves testability, since each controller and model can be validated independently before being integrated with the user interface.

From a security standpoint, every request reaching the application layer must first pass through Laravel Sanctum's token-based authentication middleware, followed by a Spatie Laravel Permission authorization check, ensuring that only verified and authorized users — whether accessing the system from the web dashboard or the mobile application — can read or modify data within the limits of their role.

IV. RESEARCH METHODOLOGY

A. Development Approach

Cutis Glow is developed using an iterative, per-module development approach across two integrated platforms. Each module (Patient, Doctor, Service/Treatment, Schedule, Booking, Medical Record, User, Role) is designed and implemented on the Laravel backend, exposed through REST API endpoints, and subsequently consumed by the corresponding screens on the Flutter mobile client, allowing backend and mobile development to proceed in parallel once the API contract for a given module has been established.

B. Tools and Technologies

Table I summarizes the technology stack used in backend and mobile development.

TABLE I
TOOLS AND TECHNOLOGIES

Layer	Technology	Version	Role / Function
Backend	Laravel	^13.8	REST API provider & core business logic
Language (BE)	PHP	^8.5	Backend server-side programming language
Database	MySQL	8.x / MariaDB	Relational data storage
Authentication & Security	Sanctum	^4.0	Token-based API authentication
RBAC	Spatie Laravel Permission	^8.2	Role & access rights management (Admin/Doctor/Patient)
Frontend	Flutter	SDK ^3.9.0	Cross-platform mobile application framework
Language (FE)	Dart	^3.9.0	Mobile application programming language

C. Data Consistency and Access Validation Strategy

As explained in Section II.C, all business logic and access-rights validation rules are centralized on the Laravel server rather than duplicated on each client, so that the web dashboard and mobile application always operate against the same validated data source and the same access rules for Admin, Doctor, and Patient.

D. Testing Method

Functional testing was carried out on both platforms to ensure that CRUD operations, authentication, and role-based access control worked as expected. On the backend side, endpoint responses were verified against the expected HTTP status codes and payload structures, for both successful and invalid requests, including attempts by a role to access modules outside its access rights. On the mobile client, manual testing was carried out on the login flow, the booking flow, and API connectivity to ensure that data entered on one platform (for example, a booking made by a patient) was correctly reflected when viewed from another platform (for example, in a doctor's schedule on the web dashboard).

V. FEATURE IMPLEMENTATION

The system is implemented across two integrated platforms. On the backend side, every API endpoint is secured using Laravel Sanctum and further restricted by Spatie Laravel Permission so that only authenticated and authorized users can access data. On the mobile side, the Flutter application uses dedicated api_service, api_client, and auth_service modules to communicate with the backend, along with local session storage to keep patients logged in on their devices.

A. Web Backend (Laravel 13, PHP 8.5, MySQL, Sanctum, Spatie Permission)

- Authentication and role-based access rights management, allowing administrators to determine which modules each role (Admin, Doctor, Patient) may access.
- A dashboard containing statistics summarizing bookings, active patients, and popular treatments.
- CRUD operations for Patient, Doctor, Service/Treatment, Schedule, and Booking data, each equipped with input validation to maintain data quality.
- A dedicated doctor module for managing schedules, confirming or rescheduling bookings, and recording treatment/medical notes.
- User and Role Management based on Spatie Laravel Permission, allowing granular configuration of access rights at the module level.
- A REST API based on Laravel Sanctum, documented and structured per resource, for consumption by the mobile application.

B. Mobile Application (Flutter — cutis_glow)

- Login and Splash screens, including token-based session storage for patients.
- A dashboard along with browsing of Service/Treatment and Doctor data.
- An online consultation booking/scheduling feature connected to doctor availability.
- Display of treatment history and personal profile, connected to the Laravel backend through a REST API, handled by dedicated api_service, api_client, and auth_service modules that separate network logic from user-interface code.

C. Security Implementation

Security is handled at several levels. At the transport and authentication level, Laravel Sanctum issues a per-user API token so that every request from the mobile client can be verified without needing to share session cookies across platforms. At the authorization level, Spatie Laravel Permission enforces role-based access checks on the backend before any CRUD operation is executed, so that a user attempting to access a module outside their role — for example, a patient trying to view another patient's medical record — will be denied regardless of what the client interface displays. At the data level, input validation and structured relationships between patients, doctors, and bookings help prevent inconsistent or conflicting scheduling data.

D. Testing and Evaluation

Functional testing was carried out on both platforms to ensure that CRUD operations, authentication, and role-based access control worked as expected, following the method described in Section IV.D.

VI. RESULTS AND DISCUSSION

The implementation results show that Cutis Glow successfully connects the Laravel-based web dashboard and the Flutter-based mobile application through a single REST API, so that booking and treatment data entered from one platform is immediately consistent on the other. This addresses the first problem stated in Section I.B, namely the need for an integrated access channel for beauty clinic management.

The three-tier, MVC-based architecture described in Section III proves effective in separating responsibilities between presentation, business logic, and data storage. This separation simplifies the implementation of new features during development, since changes to a controller's business logic do not require changes to the related view, on either the web or mobile side, as long as the API contract remains unchanged.

The role-based access control mechanism built on top of Spatie Laravel Permission also addresses the second and third problem statements. By centralizing authorization checks on the backend and enforcing a clear separation between the access rights of Admin, Doctor, and Patient, this system reduces the risk of unauthorized access to sensitive medical/treatment data — a problem that is difficult to control in paper- or spreadsheet-based manual clinic administration processes.

Compared to the manual clinic administration described in the background section, Cutis Glow offers a faster and more structured booking process, more consistent data across access channels, and visibility of medical/treatment information appropriate to each user's role. Nevertheless, the current implementation is still limited to the modules described in Section V; features such as automatic payment/payment gateway integration, patient reminder notifications, and multi-branch clinic consolidation remain open areas for further development.

VII. CONCLUSION AND RECOMMENDATIONS

A. Conclusion

Cutis Glow is a client-server-based beauty clinic management system consisting of a web dashboard (Laravel) for the Admin and Doctor roles to manage patients, schedules, services/treatments, and medical records, as well as a mobile application (Flutter) that consumes the backend REST API to make it easier for patients to access bookings and treatment history on their mobile devices.

With a three-tier architecture following the MVC pattern and role-based access control supported by Laravel Sanctum and Spatie Laravel Permission, this system is expected to address the problems of manual beauty clinic administration, such as unstructured queues, schedule conflicts, and scattered treatment histories, while also providing flexible, consistent, and role-appropriate data access across two different platforms for Admin, Doctor, and Patient.

B. Recommendations

Future development could extend the system with payment gateway integration for treatment payments, automatic booking reminder notifications, and analytical reporting features to further support clinic management decision-making. In addition, multi-branch clinic consolidation and an automated testing pipeline are recommended to further improve system reliability before deployment at a larger production scale.

REFERENCES

[1] A. Zulkifli and U. Rahmalisa, "Implementasi Framework Laravel pada Aplikasi Klinik Kecantikan Qatrunnada SkinCare," JUSIBI (Jurnal Sistem Informasi dan E-Bisnis), vol. 4, no. 1, pp. 49–59, Jan. 2022.

[2] C. Septiani, V. Valentine, V. Pranatawijaya, and N. Sari, "Penerapan Flutter pada Dashboard Admin Salon Kecantikan 'Bliss Beauty' Berbasis Mobile," JATI (Jurnal Mahasiswa Teknik Informatika), vol. 8, no. 3, pp. 3962–3966, Jun. 2024.

[3] P. W. Perdana, "Rancang Bangun Aplikasi Antrian Secara Realtime di Klinik Kecantikan Berbasis Website Menggunakan Framework Laravel," Jurnal Manajemen Informatika, vol. 11, no. 1, pp. 1–9, 2021.

[4] K. A. Seputra and L. J. E. Dewi, "A Design of Patient Registration Apps using Flutter, Laravel and Vue JS," in Proc. 4th Int. Conf. Vocational Education and Technology (IConVET 2021), Singaraja, Bali, Indonesia, Feb. 2022.

[5] N. Yasmin, M. Adri, D. Irfan, and R. Darni, "Perancangan Aplikasi Pelayanan Pasien Berbasis Mobile pada Klinik Gigi Bunga Dental Care," Jurnal Vocational Teknik Elektronika dan Informatika (Voteteknika), vol. 11, no. 3, 2023.